

LSI POO 2026 - Trabajo Práctico N° 3

Alumno/s: Caceres, Lourdes G. – Chávez, Tobías Acdel **DNI:** 45.940.137 – 47.616.254

Profesor: Mónica Ringa – Daniela Demchum

Grupo Laboratorio: 1 **TP:** 3 **Fecha de entrega:** 07/09/2026

El propósito de este trabajo práctico es interpretar diagramas de clase en UML y pasarlos al código Java, poniendo en práctica tres conceptos claves: el conocimiento entre objetos, instanciar objetos complejos y las ventajas del encapsulamiento de estructuras y objetos.

1. Conocimiento entre objetos y colaboradores

Para que un objeto pueda enviarle un mensaje a otro, es necesario que exista entre ambos una relación de conocimiento, es decir, que uno de ellos pueda nombrar al otro. Esta relación puede establecerse, entre otras formas, a través de una variable de instancia o de un parámetro recibido por un método.

Un ejemplo del primer caso se observa en la clase Circulo, cuyo atributo centro es un **colaborador** de tipo Punto. El método desplazar() no modifica directamente ningún atributo propio, sino que delega la tarea en su colaborador, respetando el principio de que cada objeto es responsable de manipular su propio estado:

```
public class Circulo {
    // atributos
    private double radio;
    private Punto centro;

    // Métodos de clase
    public void desplazar(double p_dx, double p_dy){
        this.getCentro().desplazar(p_dx, p_dy);
    }
}
```

El mismo patrón se repite en Rectangulo, que colabora con un objeto Punto para representar su origen, y en CuentaBancaria, que colabora con Persona para representar al titular de la cuenta.

El segundo caso, conocimiento mediante parámetro, se ejemplifica en el método distanciaA() de la clase Punto: el objeto receptor conoce al otro punto únicamente durante la ejecución del método, porque este último le llega como parámetro del mensaje:

```
public double distanciaA(Punto p_ptoDistante){
    double x1 = this.getX();
    double y1 = this.getY();

    double x2 = p_ptoDistante.x;
    double y2 = p_ptoDistante.y;

    double pitagoras = Math.sqrt(Math.pow((x2-x1),2) + Math.pow((y2-y1),2));
    return pitagoras;
}
```

De la misma forma, en Escuela.imprimirRecibo(Docente p_docente) el objeto Docente es un colaborador temporal: el conocimiento solo existe mientras se ejecuta ese método, y le permite a Escuela obtener los datos del docente sin necesidad de almacenarlo como atributo propio.

2. Instanciación de objetos complejos y reutilización de colaboradores

Un objeto complejo es aquel que contiene otros objetos como parte de su estructura. Por ejemplo el de CuentaBancaria, que no crea un titular “desde cero”, sino que reutiliza la clase Persona ya construida en el TP2 anterior. Esto fue posible porque la interfaz pública de Persona (nomYApe(), edad(), etc.) se mantuvo estable, lo que permitió incorporarla como colaborador en un programa nuevo sin modificar su código:

```

10 public class CuentaBancaria {
11     // variables de instancia
12     private int nroCuenta;
13     private double saldo;
14     private Persona titular;

```

Esta reutilización constituye una evidencia concreta de una de las ventajas del encapsulamiento: un objeto bien diseñado puede incorporarse como colaborador de otro sin que sea necesario conocer los detalles de su implementación interna.

3. Ocultamiento de la información y encapsulamiento

El ocultamiento de la información implica que los atributos de un objeto son privados y solo pueden modificarse a través de los métodos que el propio objeto habilita para tal fin. En CuentaBancaria, el atributo saldo es privado; ningún objeto externo puede asignarle un valor directamente, sino que debe hacerlo mediante los métodos depositar() y extraer(), que controlan la operación:

```

9 public class CuentaBancaria {
10     // variables de instancia
11     private int nroCuenta;
12     private double saldo;
13     private Persona titular;
14
15     public double depositar(double p_importe){
16         this.setSaldo(this.getSaldo() + p_importe);
17         return this.getSaldo();
18     }

```

De esta manera se evita que el saldo sea alterado por error o de forma arbitraria desde otra clase, y se centraliza la lógica de negocio dentro del propio objeto.

4. Estabilidad de la interfaz ante cambios de estructura

El encapsulamiento y el ocultamiento de información implican separar lo que un objeto hace, es decir su interfaz pública, de cómo lo hace, es decir su implementación privada. Esta separación permite modificar la implementación interna de una clase sin afectar a los objetos que la utilizan. Esto se verificó al refactorizar la clase Empleado: el atributo anioIngreso (int) fue reemplazado por fechaIngreso (Calendar), pero el método getAnioIngreso() se conservó, delegando internamente en el nuevo atributo:

```

12 public class Empleado {
13     //variables de instancia
14     private long cuil;
15     private String apellido;
16     private String nombre;
17     private double sueldoBasico;
18     private Calendar fechaIngreso; //anteriormente int anioIngreso
19
20     /*
21     public int getAnioIngreso(){ //debe seguir retornando un int
22         return this.getFechaIngreso().get(Calendar.YEAR);
23     }
24     public Calendar getFechaIngreso(){
25         return this.fechaIngreso;
26     }
27
28     /*
29     private void setAnioIngreso(int p_anio){
30         Calendar p_fecha = new GregorianCalendar(p_anio, Calendar.JANUARY, 1);
31         this.setFechaIngreso(p_fecha);
32     }

```

De esta forma, ninguna otra clase que ya utilizara getAnioIngreso() debió modificarse: el cambio de estructura quedó totalmente contenido dentro de Empleado.